Priority Scheduler in xv6

Cheolwon Park (Student ID: 2024270616)
Korea University
scott9020@korea.ac.kr

1 Implementation

xv6's default `scheduler()` was replaced with one that always picks the RUNNABLE process with the lowest numeric priority (0 = highest, 20 = lowest, default 10), breaking ties with round-robin. Two system calls — `setpriority(pid, prio)` and `getpriority(pid)` — let user programs change and query priorities at runtime.

1.1 Process State

An `int priority` field was appended to `struct proc`. `allocproc()` initializes it to 10, `freeproc()` resets to 0, and `kfork()` copies the parent's value into the child right after the name, giving priority inheritance. A second field `sched_last` was added to `struct cpu` to remember the last-dispatched `proc[]` index so that the scheduler can rotate its scan start.

1.2 System Calls

`SYS_setpriority` (22) and `SYS_getpriority` (23) were registered in `kernel/syscall.h` and wired through `syscall.c` (externs + `syscalls[]`), `user/usys.pl` (trap stubs), and `user/user.h` (C prototypes). The handlers in `sysproc.c` read arguments with `argint` and delegate to two thin helpers in `proc.c` — `ksetpriority` and `kgetpriority` — which iterate `proc[]` under per-process locks to locate the target pid; `ksetpriority` rejects priorities outside [0, 20] before scanning. Both return `-1` on failure. Prototyping the helpers in `defs.h` keeps `proc[]` encapsulated in `proc.c`.

1.3 Scheduler

The round-robin loop was replaced with a single-pass "best-so-far" search over a rotated scan order starting at `(sched_last + 1) mod NPROC`. A `best` pointer keeps its lock held; when a visited entry is RUNNABLE and strictly lower-priority than `best`, the previous candidate's lock is released and the new entry replaces it. After the scan the winner is dispatched with the lock still held (state to `RUNNING`, `swtch`), preserving xv6's locking invariants. Rotating the scan start is what makes tie-breaking genuine round-robin: without it, three children that all start at priority 10 would keep re-selecting the lowest-index tied process and starve its siblings — an issue actually observed during development and fixed by adding `sched_last`.

1.4 Files Modified

File	Modification	Step
`kernel/proc.h`	Added `priority` to `struct proc`, `sched_last` to `struct cpu`	1
`kernel/proc.c`	Priority init/reset/inherit; rewrote `scheduler()`; `ksetpriority`/`kgetpriority`	2, 7
`kernel/defs.h`	Prototypes for the two helpers	5
`kernel/syscall.h`	`SYS_setpriority=22`, `SYS_getpriority=23`	3
`kernel/syscall.c`	Externs and `syscalls[]` entries	6
`kernel/sysproc.c`	`sys_setpriority`, `sys_getpriority`	5
`user/usys.pl`, `user/user.h`	Trap stubs and C prototypes	4
`Makefile`	Added `$U/_priority_test` to `UPROGS`	Test

The kernel was rebuilt and `priority_test` run under QEMU. All three subtests pass: setpriority/ getpriority with range checks (Test 1), priority inheritance across `fork()` (Test 2), and three children with priorities 1/10/19 all completing (Test 3).

2 Theoretical Background with Questions

2.1 With your priority scheduler, what happens to a low-priority process if high-priority processes keep arriving? How does this relate to the convoy effect from FCFS that we studied in theory?
Under a strict priority scheduler a low-priority process can be starved indefinitely if higher-priority processes keep arriving: the best-so-far scan always picks one of them first, and the low-priority process is never dispatched. This is the priority-scheduling analogue of the FCFS **convoy effect** — in both cases some processes wait unbounded time for CPU — but the cause differs. FCFS punishes circumstance (arrival order behind a long burst), while priority scheduling punishes a property (the priority value), so the disadvantage persists across workloads. The classical remedy is **aging**: the priority of a waiting process is gradually boosted over time, guaranteeing that any starved process eventually climbs to the top of the queue.

2.2 Run 3 `spin` processes with priorities 1, 10, and 19 (using `setpriority` in a wrapper). Compare the scheduling behavior with the round-robin behavior you observed in Lab Exercise 3. What is different?
In Week 6 Lab Ex. 3, three `spin` processes under round-robin interleaved their output nearly evenly — each got a tick in turn. The equivalent experiment here is `priority_test`'s Test 3, which forks three CPU-bound children and sets them to priorities 1/10/19 via `setpriority` — the same workload structure as three prioritized `spin`s. Under the new scheduler HIGH becomes the strictly best candidate once it has lowered its own priority to 1 and then monopolizes the CPU until it exits; MED runs to completion next, then LOW. Two conditions must hold before this ordering is observable on the finished line: the burst must be long enough to cross a quantum boundary (~100 ms, set in `clockintr`), and the tie-break must rotate. With the shipped `burn(`5×10^6) on `CPUS=1` each child completes in one quantum and the order is simply fork order (LOW→MED→HIGH); raising burst to 5×10^7 exposes preemption and the expected HIGH→MED→LOW ordering appears. The key difference from round-robin is qualitative; round-robin interleaves forever, whereas priority *serializes* by priority class.

2.3 SJF is considered optimal for minimizing average waiting time. How would you use priority scheduling to approximate SJF? What information would you need?
SJF is already a priority scheduler in disguise, with priority equal to (the inverse of) the predicted next CPU burst. To approximate it, the kernel maintains a per-process running estimate of the next burst length and remaps it onto the 0–20 priority scale before each scheduling decision. The estimate comes from observed history, typically via exponential averaging $\tau_{n+1} = \alpha \cdot t_n + (1-\alpha)\cdot\tau_n$, where t_n is the most recently measured CPU burst. Measurement requires recording, at every context switch, how long the process ran before yielding or blocking; only CPU bursts count, not I/O waits. The data needed is therefore a per-process burst history stored inside `struct proc` alongside the priority field. Pure SJF is brittle, though: it relies on estimates tracking reality, and heavily I/O-bound processes with very short bursts would dominate the CPU — which is why real systems combine SJF-like bias with aging and multilevel feedback queues rather than using it alone.

→ Raising the burst time actually produced the expected results, which initially confused me. But investigating why gave me a deeper understanding of how Round Robin behaves.